

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 4

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение «Бронирование столиков в ресторанах» на микросервисы, придерживаясь концепции database-per-service. Спроектировать микросервисную архитектуру, описать способы взаимодействия сервисов, разделение баз данных, спроектировать эндпоинты для межсервисного обмена в формате OpenAPI. Сформировать план перехода по паттерну Strangler Fig.

Ход работы

1. Анализ текущего монолита

Существующее приложение (результат ЛР1) реализовано как монолит на связке Express + TypeORM + PostgreSQL. Все девять таблиц базы данных находятся в одной схеме, все эндпоинты обрабатываются одним сервером на порту 3000.

Таблицы монолита:

users — пользователи

restaurants — рестораны

cuisines — справочник кухонь

restaurant_cuisines — связь M: N ресторанов и кухонь

tables — столики

menu_items — блюда меню

restaurant_photos — фотографии ресторанов

bookings — бронирования

reviews — отзывы

Основные группы эндпоинтов: аутентификация (регистрация, вход, обновление токенов), профиль пользователя, каталог ресторанов с фильтрацией, меню, столики, бронирования с проверкой доступности, отзывы, фотографии.

Недостатки монолитного подхода: невозможность независимого масштабирования, единая точка отказа, сложность внесения изменений в отдельные модули.

2. Декомпозиция на микросервисы

На основе анализа предметной области и структуры базы данных выделены три микросервиса, каждый из которых владеет собственным набором таблиц и предоставляет изолированный функционал.

Таблица 1 — Состав микросервисов

Микросервис	Порт	База данных	Таблицы	Функционал
auth-service	8001	auth_db	Users	Регистрация, вход, обновление токенов, профиль пользователя
restaurant-service	8002	restaurant_db	Restaurants, Cuisines, RestaurantCuisine, Tables, MenuItem, RestaurantPhotos	Управление каталогом ресторанов, кухнями, столиками, меню, фотографиями
booking-service	8003	booking_db	Bookings, Reviews	Создание и отмена бронирований, проверка занятости, отзывы, история

Обоснование границ:

auth-service изолирован по соображениям безопасности, содержит критичные данные (пароли, токены), редко изменяется.

restaurant-service объединяет операции с каталогом, которые преимущественно являются читательскими. Это позволяет кэшировать данные и масштабировать сервис независимо.

booking-service отвечает за транзакционные операции с проверкой доступности в реальном времени. Вынесен отдельно для изоляции сложной логики и пиковых нагрузок.

3. Атрибуты и связи внутри микросервисов

auth-service (база auth_db)

Содержит единственную таблицу Users с полями для аутентификации и профиля: идентификатор, email (уникальный), хэш пароля, имя, телефон, временные метки создания и обновления. Связей с другими таблицами внутри сервиса нет.

restaurant-service (база restaurant_db)

Содержит шесть таблиц, образующих каталог ресторанов. Центральная таблица — Restaurants, к которой привязаны три зависимые таблицы: Tables (столики с вместимостью и зоной), MenuItems (блюда с ценой и категорией) и RestaurantPhotos (фотографии с порядком отображения). Связь с кухнями реализована через ассоциативную таблицу RestaurantCuisine, соединяющую Restaurants и справочник Cuisines по принципу «многие-ко-многим». Все связи внутри сервиса — «один-ко-многим» от Restaurants к зависимым таблицам.

booking-service (база booking_db)

Содержит две таблицы: Bookings (бронирования) и Reviews (отзывы). Bookings хранит информацию о дате, времени, количестве гостей, статусе и ссылается на пользователя, ресторан и столик через идентификаторы без внешних ключей к другим базам. Reviews хранит оценку и текст отзыва, ссылается на пользователя и ресторан. Внутрисервисные связи между таблицами отсутствуют.

4. Межсервисное взаимодействие

Взаимодействие между микросервисами осуществляется синхронно через HTTP REST. Каждый сервис предоставляет внутренние эндпоинты, доступные только внутри сети микросервисов.

Таблица 2 — Межсервисные связи

От (booking-service)	К сервису	Эндпоинт	Назначение
Bookings.user_id	auth-service	GET /api/internal/users/{id}/exists	Проверка существования пользователя при создании брони
Reviews.user_id	auth-service	GET /api/internal/users/{id}/exists	Проверка пользователя при создании отзыва
Bookings.restaurant_id	restaurant-service	GET /api/internal/restaurants/{id}/exists	Проверка существования ресторана
Bookings.table_id	restaurant-service	GET /api/internal/tables/{id}/exists	Проверка столика и его принадлежности ресторану
Reviews.restaurant_id	restaurant-service	GET /api/internal/restaurants/{id}/exists	Проверка ресторана при создании отзыва

Обратное взаимодействие:

restaurant-service → booking-service: GET /api/internal/restaurants/{id}/stats — получение среднего рейтинга и количества отзывов для отображения на карточке ресторана.

5. Спецификация внутренних эндпоинтов (OpenAPI)

Полная спецификация внутренних эндпоинтов оформлена в формате OpenAPI 3.0 и вынесена в отдельный файл internal-api.yaml (приложен к отчёту). Файл открывается в Swagger Editor.

Краткое описание эндпоинтов:

auth-service (порт 8001):

GET /api/internal/users/{user_id}/exists — проверка существования пользователя. Возвращает { exists: true, user: {...} } (200) или { exists: false } (404).

restaurant-service (порт 8002):

GET /api/internal/restaurants/{restaurant_id}/exists — проверка ресторана. Ответ: 200 с данными или 404.

GET /api/internal/tables/{table_id}/exists?restaurant_id=... — проверка столика с дополнительной проверкой принадлежности ресторану. Ответ: 200 с данными или 404.

booking-service (порт 8003):

GET /api/internal/restaurants/{restaurant_id}/stats — получение статистики. Возвращает { rating_avg: 4.5, reviews_count: 42 } (200) или { rating_avg: 0, reviews_count: 0 } (200 при отсутствии отзывов).

Единый формат ошибок:

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "описание ошибки",
    "status": 404
  }
}
```

Все коды ответов документированы в YAML-спецификации.

6. Разделение баз данных

Каждый микросервис использует собственную базу данных на одном сервере PostgreSQL. Это обеспечивает независимость развёртывания и отсутствие разделяемых таблиц.

auth-service → база auth_db (таблица Users)

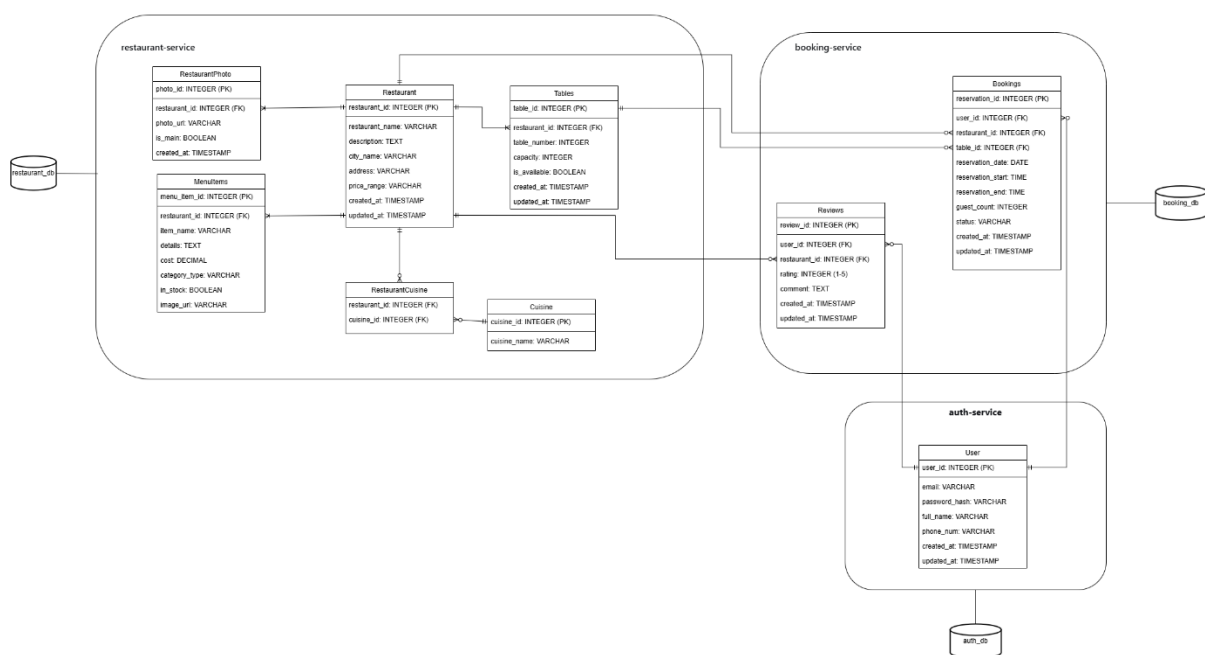
restaurant-service → база restaurant_db (таблицы Restaurants, Cuisines, RestaurantCuisine, Tables, MenuItems, RestaurantPhotos)

booking-service → база booking_db (таблицы Bookings, Reviews)

Внешние ключи между базами данных не используются. Целостность данных обеспечивается на уровне приложения через описанные выше HTTP-запросы.

7. Схема микросервисной архитектуры

Схема включает три микросервиса с их таблицами, внутренними связями (внутри сервиса) и межсервисными стрелками (через HTTP). Базы данных изображены отдельно для каждого сервиса.



На схеме отражены:

- границы микросервисов (auth-service, restaurant-service, booking-service)
- таблицы внутри каждого сервиса с атрибутами и первичными ключами
- внутрисервисные связи (Restaurants → Tables, Restaurants → MenuItems, Restaurants → Photos, RestaurantCuisine)
- межсервисные связи (Bookings → Users, Bookings → Restaurants, Bookings → Tables, Reviews → Users, Reviews → Restaurants)
- отдельные базы данных для каждого сервиса

8. План перехода от монолита к микросервисам

Для миграции предлагается паттерн Strangler Fig, позволяющий постепенно вытеснять монолит без остановки работы приложения.

Этап 1. API Gateway. Разворачивается Nginx или Express-прокси, который изначально направляет все запросы `/api/v1/*` на монолит. Клиенты продолжают работать без изменений. Этап 2. Выделение auth-service. Из монолита выносятся таблица Users и эндпоинты аутентификации (`/auth/*`, `/users/me`). Gateway начинает маршрутизировать соответствующие пути на новый сервис (порт 8001). Монолит для проверки пользователей добавляет HTTP-клиент к внутреннему эндпоинту auth-service. Этап 3. Выделение restaurant-service. Переносятся все таблицы каталога и соответствующие эндпоинты. Gateway обновляет маршрутизацию. Оставшаяся часть монолита (брони и отзывы) для валидации ресторанов и столиков использует внутренние API restaurant-service (порт 8002). Этап 4. Выделение booking-service. Последняя часть монолита (Bookings, Reviews) выносятся в отдельный сервис (порт 8003). Монолит полностью выводится из эксплуатации. Все три сервиса функционируют независимо со своими базами данных. На каждом этапе обратная совместимость клиентского API сохраняется за счёт маршрутизации через Gateway.

Вывод

В результате выполнения ДЗ4 спроектирована микросервисная архитектура для приложения бронирования столиков. Выделены три микросервиса (auth, restaurant, booking) по доменному принципу с соблюдением database-per-service. Определены синхронные HTTP-эндпоинты для межсервисного взаимодействия, специфицированные в формате OpenAPI. Разработан пошаговый план миграции с монолита по паттерну Strangler Fig. Документ содержит все требования и шаги для последующей реализации микросервисов в рамках ЛР2.